



Bônus — mBrowse avançado

Jornada DEV START · Material bônus de autoestudo · Programa START — TOTVS

Paulista Sem entrega, sem nota, sem prazo. É pra você estudar no seu ritmo, com suporte no Discord em [#duvidas-parte2-protheus](#) até **07/08**. **Precisa do ambiente Protheus** (SmartClient + AppServer + Configurador/DevStudio) para executar as telas. Mesmo sem ambiente, você pode ler, entender a lógica e escrever o código.

Sobre a tabela usada neste bônus

O projeto do Módulo 8 evoluiu em aula: a turma migrou o cadastro de exemplo de **SZ1** /Contatos para **ZA1 — Pets**, amarrada ao cadastro de Clientes (**SA1**). É a tabela que a turma já tem no ambiente, então este bônus usa ela nos exemplos, para ficar coerente com o que você já configurou:

Campo	Tipo	Tam	Contexto	Descrição
ZA1_FILIAL	C	2	Real	Filial
ZA1_COD	C	6	Real	Código do pet
ZA1_NOME	C	30	Real	Nome do pet
ZA1_ESPECIE	C	1	Real	Espécie (C=Cão, G=Gato, O=Outro)
ZA1_NASC	D	8	Real	Data de nascimento
ZA1_CLIENT	C	6	Real	Cliente (→ SA1)
ZA1_LOJA	C	2	Real	Loja do cliente (→ SA1)
ZA1_NOMCLI	C	40	Virtual	Nome do cliente (calculado)

A chave da SA1 é sempre **cliente + loja** — por isso **ZA1_NOMCLI** busca pela combinação **ZA1_CLIENT + ZA1_LOJA**, e não só pelo código do cliente.

Se a sua turma seguiu outro nome de tabela ou de campos, é só adaptar — a lógica de **mBrowse**, **aRotina**, **aColors** e filtro é **exatamente a mesma**, independente do nome da tabela.

Este material **não edita** a apostila nem os exercícios do Módulo 8 — é um complemento, para quem quer ir além do que foi (ou seria) dado em aula.

1. De AxCadastro para mBrowse: por que existe o mBrowse?

Se você já fez o CRUD de **ZA1** com **AxCadastro**, ele já funciona. Então **por que trocar** para **mBrowse**? Porque no mundo real você vai precisar de mais **controle** sobre a tela.

	AxCadastro	mBrowse
Código	Mínimo	Mais detalhado
Controle da interface	Baixo	Alto
Legendas coloridas	✗ Não	✓ Sim
Filtros dinâmicos	Limitado	✓ Completo
Colunas customizadas	✗ Não	✓ Sim
Uso típico	Protótipos, tabelas simples	Rotinas de produção



A regra prática: use **AxCadastro** para **aprender e prototipar**. Use **mBrowse** para o **sistema real** — quando precisa de legendas, filtros, colunas sob medida e comportamento profissional.

A boa notícia: a base é a mesma. **cCadastro**, **aRotina** e o dicionário continuam valendo. Você só **troca a função de abertura** — e ganha superpoderes.

2. Refazendo o CRUD de ZA1 com mBrowse

A versão AxCadastro (ponto de partida)

```
// =====  
// STPET001.PRW  
// Cadastro de Pets (AxCadastro)  
// =====  
  
#include "protheus.ch"  
  
USER FUNCTION STPET001()  
  
    PRIVATE cCadastro := "Pets"  
  
    dbSelectArea("ZA1")  
    dbSetOrder(1)  
  
    AxCadastro("ZA1", "Pets", , "1", , , , .F.)  
  
RETURN NIL
```

A versão mBrowse (o que este bônus foca)

```
// =====
// STPET002.PRW
// Cadastro de Pets (mBrowse)
// =====

#include "protheus.ch"

USER FUNCTION STPET002()

    LOCAL cFiltro := ""

    PRIVATE cCadastro := "Pets"
    PRIVATE aRotina := {;
        {"Pesquisar", "AxPesqui", 0, 1},;
        {"Visualizar", "AxVisual", 0, 2},;
        {"Incluir", "AxInclui", 0, 3},;
        {"Alterar", "AxAltera", 0, 4},;
        {"Excluir", "AxDeleta", 0, 5};
    }

    dbSelectArea("ZA1")
    dbSetOrder(1)

    mBrowse(1, 1, 22, 75, "ZA1", , , , , , , , , , , cFiltro)

    RETURN NIL
```

Note que o **aRotina** continua igual ao do **AxCadastro** — as funções **AxInclui** / **AxAltera** / etc. são reaproveitadas. O que muda é a **função de abertura** (**mBrowse** no lugar de **AxCadastro**).

aRotina — cada linha, explicada

```
PRIVATE aRotina := {;
    {"Pesquisar", "AxPesqui", 0, 1},;
    {"Visualizar", "AxVisual", 0, 2},;
    {"Incluir", "AxInclui", 0, 3},;
    {"Alterar", "AxAltera", 0, 4},;
    {"Excluir", "AxDeleta", 0, 5};
}
```

Cada linha tem **4 elementos**: **{"Texto do botão", "Função", 0, TipoDaOperação}**. Os tipos: **1**=Pesquisar, **2**=Visualizar, **3**=Incluir, **4**=Alterar, **5**=Excluir, **6**=customizado (você escreve a

função).

3. Legendas coloridas (aColors)

Uma das funcionalidades mais poderosas do `mBrowse` : **pintar as linhas** conforme regras de negócio. Imagine a lista de pets — você quer que o usuário **veja na hora** quais são filhotes, adultos ou idosos, sem precisar abrir cada registro:

```
LOCAL aColors := {;
    {"ZA1->ZA1_NASC >= dDataBase - 365",      "BR_GREEN"},;    // filhote: até 1 ano
    {"ZA1->ZA1_NASC >= dDataBase - (365 * 8)", "BR_YELLOW"},;  // adulto: 1 a 8 anos
    {".T.",                                     "BR_BLUE"};      // idoso: 8+ anos
    (padrão)
};

mBrowse(1, 1, 22, 75, "ZA1", , , , , aColors, , , , .F., , , cFiltro)
```

⚠ **A ordem importa!** As regras são avaliadas **de cima para baixo** — a **primeira** que for verdadeira define a cor da linha. Por isso a regra `".T."` (sempre verdadeira) fica **por último**, funcionando como cor padrão.

Cores disponíveis

Constante	Cor	Constante	Cor
<code>BR_RED</code>	Vermelho	<code>BR_CYAN</code>	Ciano
<code>BR_GREEN</code>	Verde	<code>BR_MAGENTA</code>	Magenta
<code>BR_YELLOW</code>	Amarelo	<code>BR_BLACK</code>	Preto
<code>BR_BLUE</code>	Azul	<code>BR_WHITE</code>	Branco

4. Filtro pré-definido + botão para remover o filtro

Um recurso que aparece direto em rotinas reais: abrir a tela **já filtrada** (por exemplo, só os pets de uma espécie), com um **botão** para o usuário voltar a ver tudo quando quiser.

Passo 1 — o filtro pré-definido

```
// Mostrar apenas cães (ZA1_ESPECIE == "C")  
LOCAL cFiltro := "ZA1->ZA1_ESPECIE == 'C'"  
  
mBrowse(1, 1, 22, 75, "ZA1", , , , , , , , , , .F., , , cFiltro)
```

Passo 2 — o botão que remove o filtro

A ideia: um botão customizado (tipo 6) chama uma função que **reabre** o browse sem filtro:

```

// =====
// STPET002.PRW
// Cadastro de Pets (mBrowse) com filtro pré-definido
// e botão para remover o filtro
// =====

#include "protheus.ch"

USER FUNCTION STPET002()

    LOCAL cFiltro := "ZA1->ZA1_ESPECIE == 'C'"    // filtro inicial: só cães

    PRIVATE cCadastro := "Pets (somente cães)"
    PRIVATE aRotina := {;
        {"Pesquisar",    "AxPesqui",    0, 1},;
        {"Visualizar",   "AxVisual",    0, 2},;
        {"Incluir",      "AxInclui",    0, 3},;
        {"Alterar",      "AxAlterar",    0, 4},;
        {"Excluir",      "AxDeleta",    0, 5},;
        {"Ver todos",    "U_STPET002ALL", 0, 6}; // ← botão customizado que remove o
filtro
    }

    dbSelectArea("ZA1")
    dbSetOrder(1)

    mBrowse(1, 1, 22, 75, "ZA1", , , , , , , , , , .F., , , cFiltro)

RETURN NIL

// =====
// Reabre a ZA1 sem filtro
// =====
USER FUNCTION STPET002ALL()

    PRIVATE cCadastro := "Pets (todos)"
    PRIVATE aRotina := {;
        {"Pesquisar",    "AxPesqui",    0, 1},;
        {"Visualizar",   "AxVisual",    0, 2},;
        {"Incluir",      "AxInclui",    0, 3},;
        {"Alterar",      "AxAlterar",    0, 4},;
        {"Excluir",      "AxDeleta",    0, 5};
    }

    dbSelectArea("ZA1")
    dbSetOrder(1)

    mBrowse(1, 1, 22, 75, "ZA1", , , , , , , , , , .F., , , "") // cFiltro vazio =
sem filtro

RETURN NIL

```

💡 O botão “**Ver todos**” simplesmente chama uma segunda **USER FUNCTION** que abre a mesma tabela, com o mesmo **aRotina**, mas passando **cFiltro** vazio. É uma técnica simples e cobre a maioria dos casos reais de “filtro + botão de limpar”.

Combinando tudo: filtro + legendas + botão

```
#include "protheus.ch"

USER FUNCTION STPET002()

LOCAL cFiltro := "ZA1->ZA1_ESPECIE == 'C'"
LOCAL aColors

PRIVATE cCadastro := "Pets (somente cães)"
PRIVATE aRotina := {;
    {"Pesquisar", "AxPesqui", 0, 1},;
    {"Visualizar", "AxVisual", 0, 2},;
    {"Incluir", "AxInclui", 0, 3},;
    {"Alterar", "AxAltera", 0, 4},;
    {"Excluir", "AxDeleta", 0, 5},;
    {"Ver todos", "U_STPET002ALL", 0, 6};
}

aColors := {;
    {"ZA1->ZA1_NASC >= dDataBase - 365", "BR_GREEN"},;
    {"ZA1->ZA1_NASC >= dDataBase - (365 * 8)", "BR_YELLOW"},;
    {"T.", "BR_BLUE"};
}

dbSelectArea("ZA1")
dbSetOrder(1)

mBrowse(1, 1, 22, 75, "ZA1", , , , , aColors, , , , .F., , , cFiltro)

RETURN NIL
```

5. Campo virtual: ZA1_NOMCLI

O **ZA1_NOMCLI** busca o nome do cliente na **SA1**, pela combinação **ZA1_CLIENT + ZA1_LOJA** — a chave completa do cliente. No **X3_RELACAO** do campo, configurado como **Virtual**:

```
POSICIONE("SA1", 1, xFilial("SA1") + M->ZA1_CLIENT + M->ZA1_LOJA, "A1_NOME")
```


E, para validar que o cliente informado existe, no `X3_VALID` de `ZA1_CLIENT` :

```
ExistCpo("SA1", xFilial("SA1") + M->ZA1_CLIENT + M->ZA1_LOJA, 1)
```

🧠 Lembre a diferença: `M->ZA1_CLIENT` é o valor **que o usuário está digitando agora** no formulário; `ZA1->ZA1_CLIENT` seria o valor **já gravado** no registro atual. Nas regras de validação e nos campos virtuais, você sempre trabalha com `M->` .

Para exibir o nome do cliente já na coluna do browse, sem precisar abrir o registro:

```
PRIVATE aColunas := {;  
    {"ZA1_COD",      "Código",      6},;  
    {"ZA1_NOME",     "Nome do Pet",  30},;  
    {"ZA1_ESPECIE",  "Espécie",     1},;  
    {"ZA1_NOMCLI",   "Cliente",     40};  
}  
  
mBrowse(1, 1, 22, 75, "ZA1", aColunas, , , , , aColors, , , , , .F., , , cFiltro)
```

6. Consulta padrão (SXB) — o lookup do F3 no cliente

Quando o usuário pressiona **F3** no campo `ZA1_CLIENT` , é útil abrir uma tela de consulta à `SA1` . Isso é configurado no **SXB** e apontado no campo pelo `X3_F3` . O Protheus já traz uma consulta padrão para clientes (normalmente `SA1000` ou equivalente do seu ambiente) — no SX3 do campo `ZA1_CLIENT` , basta apontar:

```
X3_F3 = "SA1010"      // ou o nome da consulta padrão de clientes do seu ambiente
```

Se quiser montar sua própria consulta (por exemplo, filtrando só clientes ativos), a estrutura é a mesma vista no Módulo 8 para o F3 da SZ1: um registro **tipo 1** definindo o alias, um **tipo 5** com a ordem de busca, um **tipo 3** por coluna do browse de consulta e um **tipo 4** com o campo de retorno.



Referência rápida

```
// --- AxCadastro (ponto de partida) ---
PRIVATE cCadastro := "Pets"
dbSelectArea("ZA1") ; dbSetOrder(1)
AxCadastro("ZA1", "Pets", , "1", , , , .F.)

// --- mBrowse básico ---
mBrowse(1, 1, 22, 75, "ZA1", , , , , , , , , , , cFiltro)

// --- aRotina: {"Texto","Funcao",0,Tipo} ---
// 1=Pesq 2=Vis 3=Inc 4=Alt 5=Del 6=Custom

// --- Legendas (avaliadas de cima p/ baixo; ".T." por último) ---
aColors := { {"ZA1->ZA1_NASC >= dDataBase-365", "BR_GREEN"},;
             {"ZA1->ZA1_NASC >= dDataBase-(365*8)", "BR_YELLOW"},;
             {"T.", "BR_BLUE"} }

// --- Filtro pré-definido + botão para remover ---
cFiltro := "ZA1->ZA1_ESPECIE == 'C'"
PRIVATE aRotina := { ..., {"Ver todos", "U_STPET002ALL", 0, 6} }
// U_STPET002ALL reabre o mesmo mBrowse com cFiltro = ""

// --- Campo virtual ZA1_NOMCLI (X3_RELACAO) ---
POSICIONE("SA1", 1, xFilial("SA1") + M->ZA1_CLIENT + M->ZA1_LOJA, "A1_NOME")

// --- Validação de referência (X3_VALID de ZA1_CLIENT) ---
ExistCpo("SA1", xFilial("SA1") + M->ZA1_CLIENT + M->ZA1_LOJA, 1)

// --- Colunas customizadas no browse ---
aColunas := { {"ZA1_COD","Código",6}, {"ZA1_NOME","Nome do Pet",30},
              {"ZA1_NOMCLI","Cliente",40} }
mBrowse(1, 1, 22, 75, "ZA1", aColunas, , , , , aColors, , , , .F., , , cFiltro)
```

Campo

Papel

ZA1_CLIENT + ZA1_LOJA

Chave estrangeira → SA1 (cliente + loja)

ZA1_NOMCLI

Campo Virtual — nome do cliente, calculado via
POSICIONE

M->campo

Valor em edição no formulário

ZA1->campo

Valor já gravado no registro atual

🎯 Fez o CRUD com `mBrowse` , as legendas e o filtro com botão de remover? Você já domina os recursos de `mBrowse` mais usados em rotinas reais de Protheus. Dúvidas: `#duvidas-parte2-protheus` . 🔍